

Functional Architecture — architecture-model-standard

Purpose

This document describes the functional decomposition of the Architecture Model Standard into 10 capabilities realized by 24 components. It covers the two major decomposed subsystems (CORE with 7 sub-components, MANIFEST with 8 sub-components), seven standalone F-blocks, and the cross-cutting concerns that bind them together. All APIs use typed returns — no raw dicts cross component boundaries.

Top-Level F-Block Decomposition

F-Block	Capability	Realizing Component(s)
CAP-F1	Model Parsing & Validation	COMP-CORE (□ PARSER, VALIDATOR, TYPES)
CAP-F2	Reality Manifest Generation	COMP-MANIFEST (□ SCANNER, BLOCKS, METRICS, INTERFACES, BODY-HINTS, TEST-ANALYZER, GENERATOR, TYPES)
CAP-F3	Model Slicing & Diffing	COMP-CORE (□ SLICER, DIFFER, MERGER, DECOMPOSER)
CAP-F4	CLI Operations	COMP-CLI
CAP-F5	Configuration Management	COMP-CONFIG
CAP-F6	Schema Specification	COMP-SPEC
CAP-F7	Model Extraction	COMP-EXTRACT
CAP-F8	Domain Profiles	COMP-PROFILES
CAP-F9	Shared Utilities	COMP-UTILS
CAP-F10	Auto-Enrichment	COMP-ENRICH

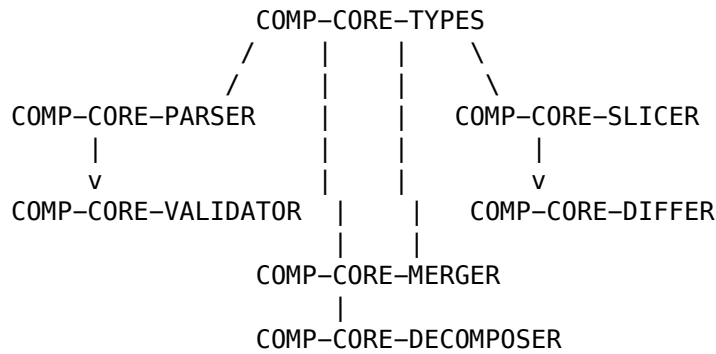
CORE Subsystem (CAP-F1 + CAP-F3)

COMP-CORE decomposes into 7 sub-components spanning two capabilities: parsing/validation (F1) and slicing/diffing (F3).

Sub-Component Table

Sub-Component	ID	Responsibility
Parser	COMP-CORE-PARSER	YAML □ ArchitectureModel dataclass conversion
Validator	COMP-CORE-VALIDATOR	Structural checks, scoring (0–100), orphan detection
Slicer	COMP-CORE-SLICER	Subset extraction by F-block, layer, or artifact
Differ	COMP-CORE-DIFFER	Version comparison with typed change records
Merger	COMP-CORE-MERGER	Compose enriched models from manifest + base model
Decomposer	COMP-CORE-DECOMPOSER	Test-affinity subsystem identification
Types	COMP-CORE-TYPES	ArchitectureModel, ValidationResult, entity dataclasses

Internal Dependency Diagram



All sub-components depend on TYPES for shared dataclasses. VALIDATOR consumes parsed models. DIFFER operates on sliced models. MERGER and DECOMPOSER sit at the top of the dependency chain.

Key Function Signatures

COMP-CORE-PARSER

`load_model(path: Path) -> ArchitectureModel`

COMP-CORE-VALIDATOR

`validate_model(model: ArchitectureModel) -> ValidationResult`

ValidationResult: score (0-100), issues: list[ValidationIssue], is_valid: bool

COMP-CORE-SLICER

`slice_by_fblock(model: ArchitectureModel, fblock_id: str) -> ArchitectureModel`

`slice_by_layer(model: ArchitectureModel, layer_id: str) -> ArchitectureModel`

COMP-CORE-DECOMPOSER

`decompose(model: ArchitectureModel) -> list[Subsystem]`

MANIFEST Subsystem (CAP-F2)

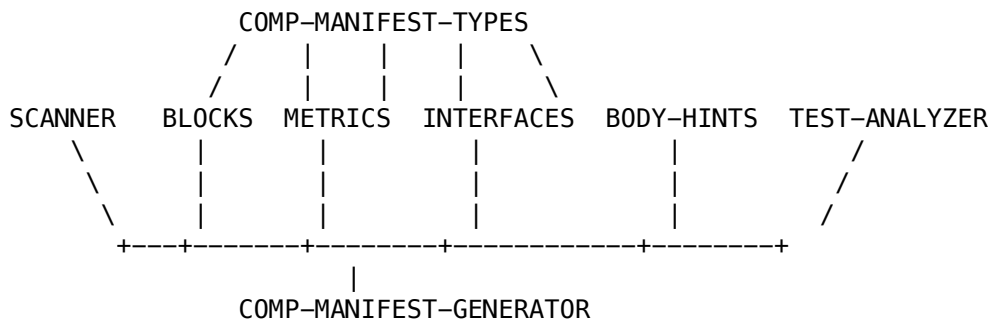
COMP-MANIFEST decomposes into 8 sub-components that perform AST scanning, metric computation, and test analysis to produce a ground-truth Manifest object.

Sub-Component Table

Sub-Component	ID	Responsibility
Scanner	COMP-MANIFEST-SCANNER	AST scanning of Python source files
Blocks	COMP-MANIFEST-BLOCKS	Functional block discovery from subpackages
Metrics	COMP-MANIFEST-METRICS	Code metrics (LOC, function count, complexity)
Interfaces	COMP-MANIFEST-INTERFACES	Interface detection from function signatures and classes

Sub-Component	ID	Responsibility
Body Hints	COMP-MANIFEST-BODY-HINTS	Trivial function implementation extraction
Test Analyzer	COMP-MANIFEST-TEST-ANALYZER	Test file analysis and contract extraction
Generator	COMP-MANIFEST-GENERATOR	Orchestrates all sub-components into final Manifest
Types	COMP-MANIFEST-TYPES	Manifest, ModuleInfo, BlockManifest, MetricsResult dataclasses

Internal Dependency Diagram



GENERATOR is the sole orchestrator — it calls all other sub-components and assembles the final Manifest.

Key Function Signatures

COMP-MANIFEST-GENERATOR

`generate_manifest(project_root: Path) -> Manifest`

COMP-MANIFEST-SCANNER

`scan_file(root: Path, filepath: Path) -> ModuleInfo`

COMP-MANIFEST-BLOCKS

`process_block(root: Path, block_id: str, block_def: dict) -> BlockManifest`

COMP-MANIFEST-METRICS

`compute_metrics(root: Path, config: ProjectConfig) -> MetricsResult`

COMP-MANIFEST-INTERFACES

`detect_interfaces(modules: list[ModuleInfo]) -> list[InterfaceEdge]`

COMP-MANIFEST-BODY-HINTS

`extract_file_hints(filepath: Path) -> list[FunctionSignature]`

COMP-MANIFEST-TEST-ANALYZER

`analyze_test_file(test_file: Path) -> TestAnalysisResult`

CLI (CAP-F4)

COMP-CLI provides the `architecture-model` command-line interface. It delegates to CORE, MANIFEST, CONFIG, and EXTRACT — contains no domain logic itself.

Commands: `init`, `validate`, `slice`, `diff`, `stats`, `impact`, `manifest`, `extract`, `context`, `query`, `generate`.

Configuration Management (CAP-F5)

COMP-CONFIG handles auto-discovery of project structure and configuration loading.

```
discover_config(root: Path) -> tuple[ProjectConfig, DiscoveryReport]
```

Discovers source root (`src-layout`, `flat-layout`, `lib-layout`), enumerates subpackages, and produces a `ProjectConfig` used by MANIFEST and CORE.

Schema Specification (CAP-F6)

COMP-SPEC owns the JSON Schema that defines the model format: 7 entity types (actors, capabilities, behaviors, interfaces, constraints, layers, components) and 8 relationship types. Used by VALIDATOR for compliance checks.

Model Extraction (CAP-F7)

COMP-EXTRACT converts LLM-generated Tier 1 artifacts into validated `ArchitectureModel` instances. Bridges the gap between free-form LLM output and the structured schema.

Domain Profiles (CAP-F8)

COMP-PROFILES extends the base schema with domain-specific entity kinds, properties, and validation rules.

```
load_profile(name: str) -> DomainProfile
# Profiles: software (default), controls, mechanical, electrical
```

Shared Utilities (CAP-F9)

COMP-UTILS provides file discovery and exclusion pattern logic shared by MANIFEST and CORE. No domain semantics — pure infrastructure.

Auto-Enrichment (CAP-F10)

COMP-ENRICH bridges the model and manifest subsystems. It takes a base `ArchitectureModel` and augments components with AST-derived data: function signatures, constants, and test contracts.

```
enrich_model(model: ArchitectureModel, project_root: Path) -> ArchitectureModel
```

Self-model enrichment stats: 83 signatures, 11 constants, 109 test contracts across 24 components.

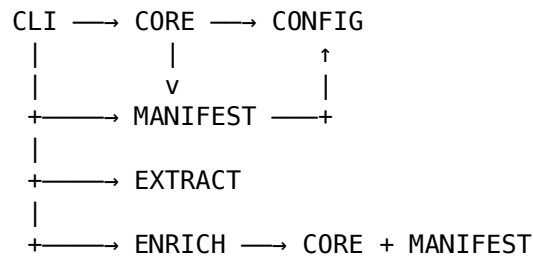
Cross-Cutting Concerns

Domain profiles (COMP-PROFILES) affect both parsing and validation — the parser must accept profile-extended entity kinds, and the validator must apply profile-specific rules.

Shared utilities (COMP-UTILS) are consumed by both MANIFEST (file discovery for AST scanning) and CORE (file resolution for model loading).

Enrichment (COMP-ENRICH) is the bridge between MANIFEST (which produces AST data) and CORE (which owns the model). It depends on both subsystems but neither depends on it.

Dependency direction:



PROFILES → CORE (parser + validator)

UTILS → CORE + MANIFEST

SPEC → CORE (validator)

No circular dependencies exist.

Schema Version

Current schema version: **v1.5** — adds `ObservabilityContract` on components, alongside `FunctionSignature`, `Constant`, and `TestContract` introduced in v1.4. These enrichment-tier fields enable blind regeneration from model alone.

Navigational Diagrams

The following diagrams show entity relationships at different zoom levels:

- **Full model:** nav-diagram.png
- **CAP-F1 focus (Parsing & Validation):** focused-cap-f1.png
- **CAP-F2 focus (Manifest Generation):** focused-cap-f2.png
- **CAP-F8 focus (Domain Profiles):** focused-cap-f8.png